



MERN Stack Final Revision Cheat Sheet

Interview: Tomorrow (Dec 27) | Format: MCQ + Coding (1hr)

JavaScript Quick Reference

Variables

Type	Scope	Redeclare	Reassign	Hoisted
var	Function	✓	✓	Yes (undefined)
let	Block	✗	✓	TDZ
const	Block	✗	✗	TDZ

Falsy Values (6)

false, 0, "", null, undefined, NaN

== vs ===

- == → Type coercion ("5" == 5 → true)
- === → Strict, no coercion ("5" === 5 → false)

Closures

Function that remembers outer scope variables after outer function returns.

```
function outer(x) {  
  return function inner(y) { return x + y; }  
}
```

Arrow Functions

- No own this (inherits from parent)
- No arguments object
- Can't be used as constructor

Spread & Rest

```
// Spread: Expand  
const merged = [...arr1, ...arr2];  
const clone = { ...obj };  
  
// Rest: Collect  
function sum(...nums) { }  
const [first, ...rest] = arr;
```

Destructuring

```
const { name, age = 0 } = user; // Object
const [a, , b] = arr;           // Array (skip)
```

Optional Chaining & Nullish Coalescing

```
user?.address?.city // undefined if any null/undefined
value ?? 'default'   // only for null/undefined (not 0, '')
```

Array Methods

Method	Returns	Mutates
map()	New array	No
filter()	New array	No
reduce()	Single value	No
find()	Element	No
sort()	Same array	Yes
push/pop	Length/Element	Yes

Promises & Async/Await

```
// Promise
promise.then().catch().finally()
Promise.all([]) // All must succeed
Promise.race([]) // First to complete

// Async/Await
async function fn() {
  try { await promise; }
  catch (e) { }
}
```

Event Loop Order

1. **Sync code** (Call stack)
2. **Microtasks** (Promises, queueMicrotask)
3. **Macrotasks** (setTimeout, events)

Debounce vs Throttle

- **Debounce**: Wait until activity STOPS, then execute
- **Throttle**: Execute at most once per time period

Node.js Quick Reference

Key Concepts

- JavaScript runtime on V8
- Single-threaded + Event loop
- Non-blocking I/O

Core Modules

```
const fs = require('fs');      // File system
const path = require('path');  // File paths
const http = require('http');  // Web server
require('dotenv').config();    // Env vars → process.env.VAR
```

Common Path Methods

```
path.join('/a', 'b', 'c')    // '/a/b/c'
path.basename('/a/b.txt')    // 'b.txt'
path.extname('file.txt')     // '.txt'
```

NPM Commands

```
npm init -y          # Initialize
npm i express        # Install
npm i -D jest        # Dev dependency
npm run dev          # Run script
```

Express.js Quick Reference

Basic Setup

```
const app = express();
app.use(express.json());    // Parse JSON body
app.use(cors());           // Enable CORS
app.listen(3000);
```

Routes

```
app.get('/users', (req, res) => {})
app.get('/users/:id', (req, res) => {
  const { id } = req.params;    // Route params
})
app.post('/users', (req, res) => {
  const data = req.body;        // Request body
})
// req.query → Query string ?page=1
```

Middleware

```
// Custom
const logger = (req, res, next) => {
  console.log(req.method);
  next(); // Must call next()!
};

// Error handler (4 params)
app.use((err, req, res, next) => {
  res.status(500).json({ error: err.message });
});
```

Response Methods

```
res.json({ data })           // Send JSON
res.status(201).json({})     // With status
res.send('text')             // Send text
res.redirect('/url')         // Redirect
```

HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Server Error

MongoDB Quick Reference

Terminology

MongoDB	SQL
Collection	Table
Document	Row
Field	Column

_id	Primary Key
-----	-------------

Mongoose Schema

```
const schema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, unique: true },
  age: { type: Number, min: 0 },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  posts: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Post' }]
}, { timestamps: true });
```

CRUD Operations

```
// Create
await Model.create({ name, email });

// Read
await Model.find({ active: true });
await Model.findById(id);
await Model.findOne({ email });
await Model.find().sort('-createdAt').limit(10);

// Update
await Model.findByIdAndUpdate(id, { name }, { new: true });

// Delete
await Model.findByIdAndDelete(id);
```

Query Operators

```
{ age: { $gt: 18 } }      // Greater than
{ age: { $gte: 18 } }     // Greater or equal
{ age: { $lt: 18 } }      // Less than
{ age: { $in: [20, 25] } } // In array
{ $or: [{ a: 1 }, { b: 2 }] } // OR
```

Update Operators

```
{ $set: { name: 'New' } } // Set field
{ $inc: { count: 1 } }     // Increment
{ $push: { tags: 'new' } } // Add to array
{ $pull: { tags: 'old' } } // Remove from array
```

Population

```
await Post.findById(id).populate('author', 'name email');
```

React.js Quick Reference

JSX Rules

- Use `className` not `class`
- Use `htmlFor` not `for`
- Self-close tags: `<img />`
- Single root (use `<>...</>` for fragments)

useState

```
const [value, setValue] = useState(initialValue);
setValue(newValue);           // Direct
setValue(prev => prev + 1);    // Functional (safer)

// Objects: spread previous!
setUser(prev => ({ ...prev, name: 'New' }));
```

useEffect

```
useEffect(() => { }, []);      // Mount only
useEffect(() => { }, [dep]);    // On dep change
useEffect(() => {
  return () => { };             // Cleanup
}, []);
```

useContext

```
const Context = createContext();
<Context.Provider value={state}>
const value = useContext(Context);
```

useRef

```
const ref = useRef(null);
ref.current.focus(); // Access DOM
```

useReducer

```
const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'ACTION', payload: data });
```

useMemo & useCallback

```
const value = useMemo(() => expensive(), [deps]); // Memoize value
const fn = useCallback(() => { }, [deps]); // Memoize function
```

Custom Hook

```
function useFetch(url) {
  const [data, setData] = useState(null);
  useEffect(() => { fetch(url).then(...) }, [url]);
  return { data };
}
```

Virtual DOM

- In-memory representation of real DOM
 - Diffing algorithm compares old vs new
 - Only updates changed nodes (reconciliation)
-

Common Patterns to Remember

Debounce

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

Express Async Handler

```
const asyncHandler = fn => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
```

JWT Auth

```
jwt.sign({ id }, SECRET, { expiresIn: '7d' });
jwt.verify(token, SECRET);
```

React State Update Patterns

```
// Add to array
setItems([...items, newItem]);
```

```
// Remove from array
setItems(items.filter(i => i.id !== id));

// Update in array
setItems(items.map(i => i.id === id ? {...i, done: true} : i));
```

Interview Tips

1. **Read MCQ carefully** - Watch for tricky wording
 2. **Start coding simple** - Get basic solution first, optimize later
 3. **Think aloud** - Explain your approach
 4. **Test edge cases** - Empty arrays, null values
 5. **Don't panic** - Take a breath, you've got this!
-

Good luck tomorrow! You're prepared! 🍀